

RECOVERY GRAPH

From an AI Buildathon Idea to a Production-Grade Payment Recovery System

Autonomous payment-failure recovery with reconciliation, deterministic safety controls, causal diagnosis, and an auditable recovery ledger.

Razorpay AI Buildathon 2026 · Track 03 — AI Revenue Recovery

PROJECT JOURNEY / ENGINEERING CASE STUDY

01 — The Starting Point

Recovery Graph started with a simple question:

“What if a payment failure wasn't necessarily the end of the payment?”

In a real payment system, a `payment.failed` event does not always mean that money will never arrive. Payments can fail because of temporary bank downtime, insufficient funds, invalid payment details, UPI delays, authentication failures, timeouts, user abandonment, asynchronous completion, or duplicate webhook delivery.

The deeper problem was therefore not simply how to retry failed payments. It was how to determine whether recovery is safe, useful, and actually justified by the payment's evolving state.

That became the foundation of Recovery Graph.

02 — The Core Idea

The system was designed around a fundamental separation:

“LLMs may propose recovery intents. Deterministic software owns money, limits, retries, idempotency, and execution. Always.”

The AI layer can reason about ambiguous situations, but it does not independently decide whether money should move, whether a retry is allowed, how many times something can be retried, or whether a recovery action is eligible.

Those decisions belong to deterministic policy code. This transformed the project from an “AI agent that retries payments” into a controlled recovery system.

03 — Payment Episodes

Individual webhook events were not enough. Consider:

```
payment.failed
↓
temporary failure
↓
payment.captured
```

If the system treats the first event as final, it can initiate an unnecessary recovery payment. Recovery Graph therefore reconstructs a payment episode: the evolving story of a payment rather than one isolated event.

```
provisional_failed
↓
payment.captured
↓
captured_late
```

The safety rule is explicit: if a payment that appeared to fail is later captured, suppress recovery.

04 — From Events to State

Recovery Graph models explicit states:

```
provisional_failed
retry_pending
```

```
captured_late
final_failed
recovered
escalated
```

This makes recovery a state-transition problem instead of a collection of independent webhook handlers. The system can answer what happened, what state the payment is in, what happened before it, and what action is permitted now.

05 — Event Ingestion and Deduplication

Webhook delivery is a distributed-systems problem. Duplicate delivery is expected input, not an exceptional case.

```
Webhook
↓
Event ID
↓
Redis deduplication
↓
New event? — No —> Ignore duplicate
|
Yes
↓
Episode reconstruction
```

Redis-backed event deduplication prevents duplicate webhook delivery from becoming duplicate recovery behaviour.

06 — Diagnosis: Rules First, AI Second

A purely LLM-driven architecture would be flexible, but critical financial behaviour would become harder to constrain and audit.

Recovery Graph therefore uses a two-layer diagnosis strategy.

Known structured failures: deterministic rules handle reliably identifiable classes such as insufficient funds, invalid VPA, expired card, card do-not-honor, limit exceeded, and wrong UPI PIN.

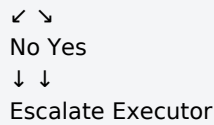
Ambiguous / long-tail failures: the LLM can assist with diagnosis when evidence is ambiguous. Its output is typed and bounded. It produces a diagnosis or recovery intent; it does not execute money movement.

07 — The Policy Gate

Diagnosis alone is not authorization. The policy layer evaluates whether a proposed action is actually allowed.

It owns recovery eligibility, retry limits, action constraints, failure-class restrictions, idempotency, and execution authorization.

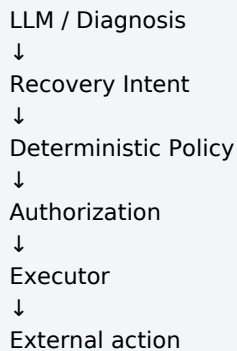
```
Diagnosis
↓
Intent
↓
Policy Gate
↓
Approved?
```



This separation is central to the safety architecture: reasoning can be probabilistic, but financial execution cannot be.

08 — Bounded Recovery Execution

The executor does not decide what should happen. It receives an action that has already passed policy validation and performs it within defined boundaries.

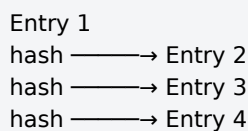


The executor is bounded, observable, idempotent, and policy-controlled.

09 — The Audit Ledger

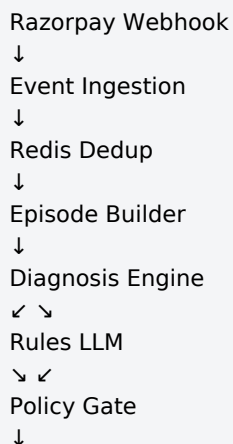
Once autonomous decisions exist, the next question is: how do we prove what happened?

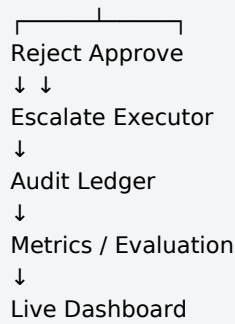
Recovery Graph uses an append-only audit ledger in PostgreSQL. Important actions and evaluation results are recorded with SHA-256 hash chaining.



The result is a tamper-evident history of system decisions covering events, diagnoses, policy decisions, recovery actions, evaluation registration, and evaluation results.

10 — The Complete Architecture





AI therefore sits inside a deterministic control system rather than replacing it.

11 — Building the Evaluation Layer

A production-looking system needs more than a successful demo. It needs evidence.

Recovery Graph includes a synthetic evaluation framework covering 200 payment episodes across ten failure classes:

```
insufficient_funds
user_abandoned
timeout_late_auth
bank_downtime
wrong_upi_pin
limit_exceeded
card_do_not_honor
expired_card
invalid_vpa
threeds_failure
```

The evaluation tests whether the diagnosis engine can correctly identify the underlying failure class.

12 — Diagnosis Results

Final diagnosis evaluation

Metric	Result
Cases	200
Correct	200 / 200
Accuracy	100.0%
Precision	1.000
Recall	1.000
Misclassified	0

This is evidence of perfect performance on the defined synthetic evaluation distribution; it should not be interpreted as proof of perfect performance on arbitrary real-world payment data.

13 — Counterfactual Evaluation

The deeper question was: does recovery actually improve outcomes?

A synthetic counterfactual benchmark was constructed to compare outcomes under treatment and control conditions.

The pre-registered evaluation specification used the hash:

08299875ab296a4a596cb1433fd8c142a86b2db4ad16dca23021c98473028c66

Measure	Result
T-learner lift	+1.95 percentage points
95% confidence interval	−5.32pp to +10.33pp
Incremental revenue	−₹33.38
RCT diff-in-means	+7.03pp

The correct interpretation is deliberately conservative: the estimated lift was +1.95 percentage points, but the confidence interval crosses zero, so the effect is not statistically conclusive.

14 — From Backend to Live System

The project evolved into an operational interface exposing Overview, Payment Episodes, Audit Ledger, Replay, Evaluation, Architecture, Scenario Injection, and Live Demo.

The UI is intended to expose internal system state rather than simply decorate the prototype.

```
Payment
↓
Episode
↓
Diagnosis
↓
Policy decision
↓
Action
↓
Audit entry
```

15 — Synthetic Scenario Injection

A controlled synthetic injection interface makes the system demonstrable without relying on live customer payments.

```
UPI Late Capture
Card Final Failure
Insufficient Funds
Duplicate Event
Invalid VPA
```

These scenarios feed events through the actual recovery pipeline: event → ingestion → deduplication → episode reconstruction → diagnosis → policy → execution → audit.

16 — The Late-Capture Demonstration

One of the most important scenarios is UPI late capture.

```
payment.failed
↓
provisional_failed
↓
payment.captured
↓
captured_late
```

Correct behaviour: WAIT + RECONCILE + SUPPRESS RECOVERY.

No unnecessary recovery payment is created. This demonstrates that the system does not confuse a temporary failure observation with a final financial outcome.

17 — Replayability

A production system should make it possible to understand how an episode reached its current state.

Recovery Graph exposes replay information so the sequence of events and decisions can be reconstructed. Debugging can therefore move from “something went wrong” to an inspectable chain of events, state transitions, decisions, and actions.

18 — Production Engineering

The project moved beyond a local prototype through Dockerized services, PostgreSQL, Redis, FastAPI, React, production frontend builds, environment-based configuration, health checks, controlled synthetic demo routes, auditability, and deterministic safety boundaries.

```
Frontend
|
↓
FastAPI Backend
|
├── PostgreSQL
|
└── Redis
```

The system was also prepared for cloud deployment.

19 — Final Validation

The final release validation was executed end-to-end after productionization.

- ✓ Git integrity
- ✓ Python compilation
- ✓ Docker services
- ✓ Backend health
- ✓ API contract
- ✓ Frontend HTTP availability
- ✓ Production frontend build
- ✓ UPI late capture
- ✓ Card final failure
- ✓ Insufficient funds
- ✓ Duplicate event
- ✓ Invalid VPA
- ✓ Episode reconstruction
- ✓ Episode detail
- ✓ Actions
- ✓ Replay

- ✓ Audit ledger
- ✓ Metrics
- ✓ 200-case diagnosis suite
- ✓ Preregistration
- ✓ Counterfactual benchmark
- ✓ Final system verification

Final diagnosis: 200/200 correct, 100% accuracy, precision 1.000, recall 1.000.

Final counterfactual: estimated T-learner lift +1.95pp; 95% CI −5.32pp to +10.33pp; RCT diff-in-means +7.03pp.

20 — What Changed From the Original Idea?

Initial framing

```
graph TD
    A[Payment failed] --> B[Ask AI]
    B --> C[Retry payment]
```

Final architecture

```
graph TD
    A[Payment event] --> B[Deduplicate]
    B --> C[Reconstruct episode]
    C --> D[Determine current state]
    D --> E[Diagnose failure]
    E --> F[Generate recovery intent]
    F --> G[Apply deterministic policy]
    G --> H[Authorize bounded action]
    H --> I[Execute]
    I --> J[Record immutable audit trail]
    J --> K[Evaluate outcome]
```

The second system is much closer to how real financial infrastructure should be designed.

21 — The Engineering Lessons

1. Events are observations, not always final truth. A failed event can later be followed by a successful capture.
2. AI should not own financial authority. AI is useful for reasoning; deterministic software should control money movement.
3. Idempotency is a feature, not an edge case. Duplicate events are expected in distributed systems.
4. Auditability should be architectural. Important decisions should produce a verifiable trail as part of the system.

5. Evaluation should be part of the product. A system should provide a way to measure whether it works.
6. Honest uncertainty is better than exaggerated success. The counterfactual result is reported with its uncertainty instead of being presented as statistical victory.

22 — The Final System

Recovery Graph ultimately became more than an AI payment-retry prototype.

- Distributed event processing
- +
- Stateful payment episodes
- +
- Rules-based diagnosis
- +
- LLM-assisted reasoning
- +
- Deterministic policy enforcement
- +
- Bounded execution
- +
- Redis idempotency
- +
- PostgreSQL persistence
- +
- Cryptographically chained auditability
- +
- Synthetic evaluation
- +
- Counterfactual analysis
- +
- Operational visualization

The central idea remained simple:

“Recover revenue without creating new financial risk.”

23 — Where the Project Ends

The project began with a question about failed payments. It ended with a broader engineering principle:

“Autonomy in financial systems should not mean giving an AI unrestricted control. It should mean giving AI useful reasoning capabilities inside a deterministic, observable, auditable control system.”

Recovery Graph is an implementation of that principle.

- The AI can reason.
- The system can reconcile.
- The policy can constrain.
- The executor can act.
- The ledger can prove what happened.

RECOVERY GRAPH

AI proposes. Code decides. The ledger remembers.

Razorpay AI Buildathon 2026